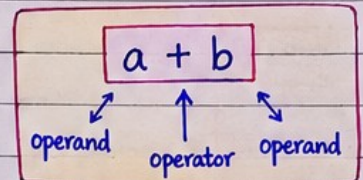


Operators and Operands in Java

- Operators (+, -, *, /, etc) are symbols that tell the compiler to perform an action.
- Operands are the values or variables on which the operation is performed.

Example : + is an operator (adds two numbers)
a and b are operands.



Expression

A combination of operands and operators is called an expression.

Example : a + b is an expression.

Sum = a + b is also an expression (with assignment).

Types of Java Operators

Type	Symbols	Description	Example
Arithmetic	+ - * / %	Used for mathematical operations.	int a = 10, b = 5; a + b = 15
Unary	+ - ++ -- !	Works with one operand.	int a = 5; a++ // a becomes 6
Binary	+ - * / %	Works with two operands.	int a = 10, b = 5; a + b = 15
Relational (Comparison)	== != > < >= <=	Compares two values. Result is true or false.	a > b // true a == b // false
Logical	&& !	Used in logical operations.	a > 0 && b > 0 !(a < b)
Bitwise	& ^ ~ << >> >>>	Works on bits of integer values.	a & b, a b
Assignment	= += -= *= /= %=	Assigns a value to a variable.	int a = 10; a += 5 // a = a + 5

* 2 Binary Operators : These works with two operands.

Operator	Operation	Example	Result
+	Addition	$5 + 3$	8
-	Subtraction	$5 - 3$	2
*	Multiplication	5×3	15
/	Division	$10 / 2$	5
%	Modulus	$10 \% 3$	1

Example of Unary :-

```
int a = 5,    a = a + 1;  (pre increment)
int b = 2,    b = b - 1;  (pre decrement)
int c = 3,    c = -c;     (unary minus)
int d = 4,    d = +d;     (unary plus)
int e = -6,   e = !e;     (logical NOT)
int w = 0,    w = ~0;     (bitwise complement)
```

Relational Operators : It use to compare two values or expression.

The result is also boolean - True or False.

List	Relational Operators	Output	Example	Result
1.	==	Equal to	$5 == 5$	True
2.	!=	Not equal to	$5 != 3$	True
3.	>	Greater than	$7 > 3$	True
4.	<	Less than	$2 < 4$	True
5.	>=	Greater than or equal	$6 >= 6$	True
6.	<=	Less than or equal	$4 <= 5$	True

Output :-

True : Returns true if the given condition is true

False : Returns false if the given condition is false

Example: $5 < 3 \rightarrow \text{False}$ (because 5 is not less than 3)

$5 <= 5 \rightarrow \text{True}$ (because 5 is equal to 5)

Note :-

Use these operators in if-else, loop conditions and to compare values.

Operators

- Ternary operator shortcut if-else statement

```
int max = (a > b) ? a : b;
```

if condition true false

```
int max = (a > 6) ? a : 6;
```

```
If a > 6 then max = a
    else max = 6
```

In normal math :

```
int n = 2 + 3 * 4 // n = 14
```

In parenthesis :

```
int n = (2 + 3) * 4 // n = 20
```

- Parenthesis () change the normal order to operations. They force Java to evaluate what just inside, no matter what.

Use to control order

→ at same level, evaluate left to right

→ Different levels → go innermost first even if it's on the right side. (most deeply nested first)

Example 1 : right side

$((2+3) * (4 + (1+1)))$ (Nested levels)

Step 1 : $(1 + 1) = 2$ (innermost)

Step 2 : $(4 + 2) = 6$

Step 3 : $(2 + 3) = 5$

Step 4 : $5 * 6 = 30$

Example 2 :

$$(1 + ((2 + 3) * (4 + 1)))$$

Step 1 : $(2 + 3) = 5$

Step 2 : $(4 + 1) = 5$

Step 3 : $5 * 5 = 25$

Step 4 : $1 + 25 = 26$



Share

Logical NOT (!)

- It **inverts** the **result**.
- If the condition is **true**, it becomes **false**.
- If the condition is **false**, it becomes **true**.

Example :

```
boolean a = true;
System.out.println(!a); // false
boolean b = false;
System.out.println(!b); // true
```

Important Notes :

- ★ It works only with boolean values (true/false).
 - ★ All the short-circuit operators (&&, ||) also return boolean value.
 - ★ In (A && B), if A is **false**, Java doesn't check B.
 - ★ In (A || B), if A is **true**, Java doesn't check B.
- (This is called **Short-Circuiting**)

Assignment Operators :

- Are used to assign value to a variable.
- Basic assignment operator is **=** (assigns value).
- Java also provides shorthand assignment operators which combine arithmetic and assignment.

Operator	Meaning	Example (a = 10)	Same as
=	Assign	a = 5;	a = 5
+=	Add and assign	a += 5;	a = a + 5
-=	Subtract and assign	a -= 3;	a = a - 3
*=	Multiply and assign	a *= 2;	a = a * 2
/=	Divide and assign	a /= 2;	a = a / 2
%=	Modulus and assign	a %= 2;	a = a % 2

Note : These are shorthand operators.

→ It also combine bitwise operators along with assignment.



Edit



Markup



Resize



Remove

> Arithmetic Operators (Unary / Binary)

```
int a = 10; int b = 3;
System.out.println(a + b);
```

↓

$a / b \rightarrow$ quotient , $a \% b \rightarrow$ remainder

* > Unary Operators

```
int a = 5;
System.out.println(++a); // 6
System.out.println(--a); // 5
System.out.println(-a); // -5
System.out.println(+a); // 5
```

$++$ = Increment (Add 1 to value)

$--$ = Decrement (Sub. 1 from value)

Type	Syntax	Explanation
Pre - increment	$++a$	Add 1 first, then use value
Post - increment	$a++$	Use the value first, then add
Pre - decrement	$--a$	Sub. 1 first, then use value
Post - decrement	$a--$	Use the value first, then Sub.

Ex.	$a = 10$	Final a	Used value
$++a$	$++a \rightarrow 11$	11	11
$a++$	$a++ \rightarrow 10$	11	10
$--a$	$--a \rightarrow 9$	9	10
$a--$	$a-- \rightarrow 10$	9	9

Note: $\text{int } a = 5 \rightarrow ++a \checkmark$ work with variable
 $\rightarrow ++5 \times$ not (constant)

$++a, --a \rightarrow$ change happen before use

$a++, a-- \rightarrow$ Use happen before change

- called **Unary** because they act on **one variable**.

* 2 **Binary Operators** : These works with two operands.

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus (Remainder)	10 % 3	1

Example of Unary Operators

```
int a = 5;
int n = ++a; // pre-increment
int y = a++; // post-increment
int z = --a; // pre-decrement
int w = a--; // post-decrement
```

Relational Operators : It used to compare two values or expressions. The result is also boolean : True or False.

List of Relational Operators

Operator	Meaning	Example	Output
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true (return true if not equal)
>	Greater than	7 > 3	true (7 is greater than 3)
<	Less than	2 < 4	true
>=	Greater than or equal to	6 >= 6	true (one condition satisfy)
<=	Less than or equal to	4 <= 5	true (one condition satisfy)

→ Use in if-else, loop conditions.

Operator Precedence

- Operator precedence means the order in which different operators are applied in an expression.
- Operators with higher precedence are evaluated first.
- If two operators have the same precedence, then associativity decides the order (left to right or right to left).

Example : `int n = 3 * 4 - 1; // 3*4 is evaluated first (higher precedence)`
`n = 12 - 1 = 11`

Operator Precedence (High to Low) and Associativity

No.	Operator Type	Operators	Associativity	Explanation
1	Postfix	<code>x++, x--</code>	Left to Right	Used after variable (e.g. <code>i++</code>)
2	Prefix	<code>++x, --x, +x, -x</code>	Right to Left	Used before variable (e.g. <code>++i</code>)
3	Multiplicative	<code>*, /, %</code>	Left to Right	Multiply, divide, remainder
4	Additive	<code>+, -</code>	Left to Right	Addition, subtraction
5	Shift	<code><<, >>, >>></code>	Left to Right	Bitwise shift
6	Relational	<code><, <=, >, >=</code>	Left to Right	Compare values (result: boolean)
7	Equality	<code>==, !=</code>	Left to Right	Check equal or not (result: boolean)
8	Bitwise AND	<code>&</code>	Left to Right	Bitwise AND
9	Bitwise XOR	<code>^</code>	Left to Right	Bitwise XOR
10	Bitwise OR	<code> </code>	Left to Right	Bitwise OR
11	Logical AND	<code>&&</code>	Left to Right	Logical AND (short-circuit)
12	Logical OR	<code> </code>	Left to Right	Logical OR (short-circuit)
13	Ternary	<code>? :</code>	Right to Left	Conditional operator (condition ? trueValue : falseValue)
14	Assignment	<code>=, +=, -=, *=, /=, % =, etc.</code>	Right to Left	Assign value to variable

Associativity of Operators

- When two operators of the same precedence appear together, Java uses associativity rules to decide the order.

- **Left to Right** : Evaluate from left side to right side.
- **Right to Left** : Evaluate from right side to left side.

Logical Operations in Java

- Logical operators are used to combine two or more conditions.
- They always give a boolean result → either **true** or **false**.
- They are useful in **decision-making statements** (if, if-else, while, etc.)

1. Logical AND (&&)

Returns **true** only if **both** conditions are **true**.

Truth Table :

A	B	A && B (Result)
true	true	true
true	false	false
false	true	false
false	false	false

Example in Java :

```
boolean a = true;
boolean b = false;
System.out.println(a && b);
// false
```

2. Logical OR (||)

Returns **true** if **at least one** condition is **true**.

Truth Table :

A	B	A B (Result)
true	true	true
true	false	true
false	true	true
false	false	false

Example in Java :

```
boolean a = true;
boolean b = false;
System.out.println(a || b);
// true
```

3. Logical NOT (!)

It **inverts** the result (opposite value).

Condition	!Condition (Result)
true	false
false	true

Example in Java :

```
boolean a = true;
System.out.println(!a);
// false
```

Summary of Logical Operators :

Operator	Name	Meaning	Result
&&	Logical AND	Both conditions must be true	true / false
	Logical OR	At least one condition must be true	true / false
!	Logical NOT	Inverts the result	true / false

Usage in Decision Making :

- Used in **if**, **if-else**, **while**, **do-while** statements.
- Helps to check multiple conditions together.

Example :

```
int age = 20;
boolean hasID = true;
if (age >= 18 && hasID) {
    System.out.println("You can enter.");
} else {
    System.out.println("You cannot enter.");
}
```